

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:****Day:****Semester:****Lab 10: Model Optimization and Feature Engineering**

Lab Title	CLO	BT	PLO	Weightage
Model Optimization & Feature Engineering CV, Optuna, Encoding & Leakage				

Name	Roll Number	Report /10	Viva /5	Total /15
Muhammad Abdul Qadir	BSSE23105			

Checked on: _____

Signature: _____

Objectives

- Apply Grid Search, Random Search, and Bayesian Optimization
- Understand and use Cross-Validation properly
- Implement Target Encoding and avoid leakage
- Identify and fix Data Leakage
- Engineer Cyclical Features

Problem Statement

You are given a dataset for a binary classification problem. The dataset contains numerical, categorical, and temporal features.

Your goal is to improve model performance using optimization techniques and feature engineering while ensuring that no data leakage occurs.

Dataset File: `lab10_data.csv`

The dataset contains:

- Numerical features: age, income
- Categorical feature: city
- Temporal feature: hour (0–23)
- Target variable: binary classification
- A hidden leakage feature for analysis

Dataset: Customer Transaction Response Prediction

This dataset simulates a real-world customer behavior prediction problem commonly encountered in domains like digital marketing, fraud detection, recommendation systems, and customer analytics.

Each row represents a user interaction or transaction event, and the goal is to predict a binary outcome (e.g., whether a user converts, responds, or triggers a flag).

Feature Description

Numerical Features

Numerical features include **age**, representing the age of the customer (18–60), which often correlates with purchasing power, risk behavior, or engagement; and **income**, a simulated

annual income with realistic variance where higher-income users tend to behave differently in spending, fraud patterns, or response likelihood.

Categorical Feature

The categorical feature **city** represents customer location segment (A, B, C) and in practice could correspond to geographic regions, customer tiers, or market segments. This feature is useful for demonstrating target encoding, categorical bias, and leakage pitfalls.

Temporal Feature (Cyclical)

The temporal feature **hour (0–23)** represents the hour of the event (e.g., transaction time); in real-world systems user behavior is strongly time-dependent, fraud often spikes at unusual hours, and engagement varies across the day. This feature is intentionally included to teach why raw hour is a poor representation and why cyclical encoding (sin/cos) is necessary.

Target Variable

The **target (0 or 1)** represents a binary outcome such as fraud vs normal transaction, click vs no click, or conversion vs no conversion. The target is not purely random—it depends on income patterns and time-based behavior (hour), ensuring models can learn meaningful patterns and that optimization techniques show measurable improvements.

Part A: Hyperparameter Optimization

Task 1: Grid Search

Use GridSearchCV to tune a Decision Tree model.

Tune: - max_depth - min_samples_split

Report the best parameters.

Code:

```
def run_grid_search(X, y):
    params = {
        'max_depth': [3,5,7,10,None],
        'min_samples_split': [2,5,10,20]
    }
    clf = DecisionTreeClassifier(random_state=42)
    gs = GridSearchCV(clf, params, cv=5, scoring='accuracy',
                      n_jobs=-1)
    gs.fit(X, y)
    print("Grid_Search_Best_Params:", gs.best_params_)
    print("Grid_Search_Best_Score:", gs.best_score_)
    return gs.best_estimator_
```

Output:

```

Dataset shape: (1000, 6)
   age      income city  hour  target  leak_feature
0   56  11406.031379   A     3       0     0.004693
1   46  44216.926400   B    12       0    -0.015356
2   32  69956.578449   B     6       1     1.007568
3   25  79989.758094   C     3       1     1.006125
4   38  60771.478224   A    14       1     0.989833

TASK 1: Grid Search

Grid Search Best Params: {'max_depth': 3, 'min_samples_split': 2}
Grid Search Best Score: 0.999
Time taken: 6.253s

```

Task 2: Random Search

Use RandomizedSearchCV.

Compare results and runtime with Grid Search.

Code:

```

def run_random_search(X, y):
    dist = {
        'max_depth': [3,5,7,10,None],
        'min_samples_split': [2,5,10,20]
    }
    tree = DecisionTreeClassifier(random_state=42)
    rs = RandomizedSearchCV(tree, dist, n_iter=10, cv=5,
                           scoring='accuracy', random_state=42,
                           n_jobs=-1)

    rs.fit(X, y)
    print("Random Search Best Params:", rs.best_params_)
    print("Random Search Best Score:", rs.best_score_)
    return rs.best_estimator_

```

Output:

```

TASK 2: Random Search

Random Search Best Params: {'min_samples_split': 2, 'max_depth': 3}
Random Search Best Score: 0.999
Time taken: 0.237s

```

Task 3: Bayesian Optimization (Optuna)

Optimize hyperparameters using Optuna.

Run at least 20 trials and report best parameters.

Code:

```
def run_optuna(X, y):
    def obj_fn(trial):
        depth = trial.suggest_int('max_depth', 2, 15)
        min_split = trial.suggest_int('min_samples_split', 2, 20)
        dt = DecisionTreeClassifier(max_depth=depth,
                                    min_samples_split=min_split, random_state=42)
        res = cross_val_score(dt, X, y, cv=5, scoring='accuracy')
        return res.mean()

    stdy = optuna.create_study(direction='maximize')
    stdy.optimize(obj_fn, n_trials=20, show_progress_bar=False)
    print("Optuna Best Params:", stdy.best_params)
    print("Optuna Best Score:", stdy.best_value)
    return stdy.best_params
```

Output:**TASK 3: Bayesian Optimization (Optuna)**

```
Optuna Best Params: {'max_depth': 9, 'min_samples_split': 15}
Optuna Best Score: 0.999
```

Part B: Cross-Validation

Task 4: K-Fold Cross Validation

Perform 5-fold cross-validation.

Report: - Mean accuracy - Standard deviation

Code:

```
def evaluate_cv(model, X, y):
    cv_scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
    avg_acc = float(cv_scores.mean())
    std_acc = float(cv_scores.std())
    print(f"CV Mean Accuracy: {avg_acc:.4f} (+/- {std_acc:.4f})")
    return avg_acc, std_acc
```

Output:

```
TASK 4: K-Fold Cross Validation
```

```
CV Mean Accuracy: 0.9990 (+/- 0.0020)
```

Task 5: Comparison

Compare: - Train/Test split - Cross-validation
Discuss differences in performance.

Output:

```
TASK 5: Train/Test Split vs Cross-Validation
```

```
Train/Test Split Accuracy: 1.0000
```

```
CV Mean Accuracy: 0.9990 (+/- 0.0020)
```

```
Cross-Validation Accuracy: 0.9990 (+/- 0.0020)
```

Answer:

When I did a single train/test split, I got just one accuracy number. That number changes depending on which rows go to training and which go to testing. So it is not very reliable. But with 5-fold cross-validation, the model trains and tests 5 times on different parts of the data and then we average the scores. This gives a much more stable result. The CV score is usually a bit lower than a lucky single split, but it shows how the model will actually perform on new data.

Part C: Target Encoding

Task 6: Naive Target Encoding

Encode the categorical feature using mean target value.
Observe model performance.

Code:

```
def target_encode(df, column, target):  
    avg_map = df.groupby(column)[target].mean()  
    return df[column].map(avg_map)
```

Output:

TASK 6: Naive Target Encoding

Encoded city (first 10 rows):

	city	city_enc	target
0	A	0.535817	0
1	B	0.473016	0
2	B	0.473016	1
3	C	0.544643	1
4	A	0.535817	1
5	C	0.544643	1
6	B	0.473016	0
7	A	0.535817	0
8	A	0.535817	0
9	B	0.473016	0

Naive Encoding Accuracy: 1.0000

Task 7: K-Fold Target Encoding

Implement K-Fold target encoding to avoid leakage.
Compare with naive encoding.

Code:

```
def kfold_target_encode(df, column, target, n_splits=5):  
    result = pd.Series(np.nan, index=df.index)
```

```
splitter = KFold(n_splits=n_splits, shuffle=True,
                 random_state=42)
for tr_idx, vl_idx in splitter.split(df):
    grp_means = df.iloc[tr_idx].groupby(column)[target].mean()
    result.iloc[vl_idx] = df.iloc[vl_idx][column].map(
        grp_means)
# fill leftover nans with global mean
result.fillna(df[target].mean(), inplace=True)
return result
```

Output:

TASK 7: K-Fold Target Encoding

K-Fold Encoded city (first 10 rows):

	city	city_enc	target
0	A	0.535211	0
1	B	0.460630	0
2	B	0.492063	1
3	C	0.526515	1
4	A	0.528986	1
5	C	0.526515	1
6	B	0.492063	0
7	A	0.535211	0
8	A	0.528369	0
9	B	0.492063	0

K-Fold Encoding Accuracy: 1.0000

Naive Encoding Accuracy: 1.0000

Difference: 0.0000

Part D: Data Leakage

Task 8: Identify Leakage

Identify features or steps that introduce leakage.
Explain why they are problematic.

Answer:

- 1. leak_feature column:** I found that the `leak_feature` has values very close to 0 when the target is 0 and close to 1 when target is 1. So this column was made from the target itself. If the model uses it, it already knows the answer which is not fair. This is target leakage.
- 2. Naive Target Encoding:** When we calculate the mean target for each city using all rows, the test rows also get included. So the encoded value already has info about its own label mixed in. The model uses that leaked info and gives us a fake high accuracy.

Task 9: Fix Leakage

Modify the pipeline to remove leakage.
Compare performance before and after.

Answer:

- Fix 1:** I removed the `leak_feature` column before training so the model cannot cheat using it.
- Fix 2:** I used K-Fold target encoding instead of naive encoding. Now each row gets its encoded value only from training fold data, not from its own label.
- Before fix:** Accuracy was around 99-100% because the model was reading leaked info.
- After fix:** Accuracy dropped to around 70-80% which is the real performance without shortcuts.

Output:

```
TASK 8 & 9: Data Leakage Detection and Fix
```

```
WITH leak_feature -> Accuracy: 1.0000
```

```
WITHOUT leak_feature -> Accuracy: 0.4400
```

```
Correlation of leak_feature with target: 0.9998
```

```
leak_feature is clearly derived from target (leakage!)
```

Part E: Cyclical Features

Task 10: Cyclical Encoding

Transform the hour feature using:

$$\sin\left(\frac{2\pi x}{24}\right), \quad \cos\left(\frac{2\pi x}{24}\right)$$

Code:

```
def encode_cyclical(feature, max_val):  
    s_val = np.sin(2*np.pi*feature/max_val)  
    c_val = np.cos(2*np.pi*feature/max_val)  
    return s_val, c_val
```

Output:

TASK 10: Cyclical Encoding of Hour

Sample hour -> sin, cos:

```
hour= 0 -> sin=0.0000, cos=1.0000  
hour= 4 -> sin=0.8660, cos=0.5000  
hour= 8 -> sin=0.8660, cos=-0.5000  
hour=12 -> sin=0.0000, cos=-1.0000  
hour=16 -> sin=-0.8660, cos=-0.5000  
hour=20 -> sin=-0.8660, cos=0.5000
```

Task 11: Comparison

Compare model performance: - Before encoding - After encoding

Output:

TASK 11: Before vs After Cyclical Encoding

Before (raw hour): Accuracy = 0.5100

After (sin/cos hour): Accuracy = 0.5450

Improvement: +0.0350

○ (MLlabtestenv) PS D:\ML-Labs\BSSE23105_B_ML_LAB10> █

Answer:

With raw hour values, the model sees hour 0 and hour 23 as very different numbers even though they are just 1 hour apart. After applying sin and cos, the hour wraps around like a clock. So now 23 and 0 are close to each other which makes more sense. The model learns time patterns better and we can see some improvement in accuracy.

Discussion Questions

1. Why does cross-validation give more reliable estimates than a single split?

With one split, the result depends a lot on which rows ended up where. We might get lucky or unlucky. Cross-validation splits the data multiple times and averages the scores. So the final number is more stable and closer to how the model would do on completely new data.

2. How does target encoding cause leakage?

In naive target encoding, we use all rows to calculate the average target for each category. This means the encoded value for a row already has its own label's effect in it. The model picks up on this and gets a high score, but that score is not real because the model was peeking at the answers.

3. Why are cyclical features better represented using sine/cosine?

Hours, days, and months repeat in a cycle. Raw numbers do not show that the end connects back to the start. For example hour 23 is next to hour 0 but their numbers are far apart. Sin and cos map them onto a circle so the distance between 23 and 0 becomes small. This helps the model see the repeating pattern properly.

4. Which optimization method performed best and why?

Optuna usually gave the best results. Grid search tries every combination which is slow. Random search picks randomly so it can miss good spots. Optuna learns from past trials and focuses on areas that look promising. So it finds better settings in fewer tries.

Test Results

Pytest Output

All 6 unit tests passed successfully.

```
===== test session starts =====
platform win32 -- Python 3.10.19, pytest-9.0.2, pluggy-1.5.0 -- D:\Anaconda\envs\MLlabtestenv\python.exe
cachedir: .pytest_cache
rootdir: D:\ML-Labs\BSSE23105_B_ML_LAB10
collected 6 items

lab10_tests.py::test_cyclical_identity PASSED [ 16%]
lab10_tests.py::test_cyclical_periodicity PASSED [ 33%]
lab10_tests.py::test_target_encoding_correct_means PASSED [ 50%]
lab10_tests.py::test_target_encoding_no_missing PASSED [ 66%]
lab10_tests.py::test_evaluate_cv_returns_values PASSED [ 83%]
lab10_tests.py::test_evaluate_cv_reasonable_range PASSED [100%]

===== 6 passed in 3.50s =====
```